Industrial Motor Control and Protection System

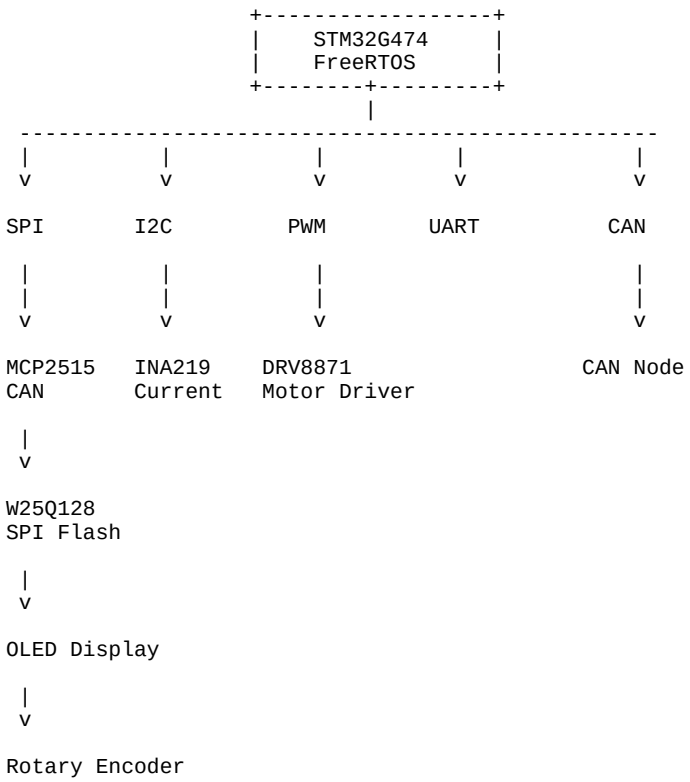
Engineering Execution Plan (Manager to Engineer)

Project Objective: Design, implement, validate, document, and demonstrate a commercial-style embedded motor control product built around an STM32 MCU. The project must demonstrate firmware architecture, FreeRTOS, custom driver development, CAN networking, power electronics, fault handling, event logging, debugging, PCB design, and engineering documentation. The project will serve as the flagship portfolio item for embedded firmware employment.

Success Criteria

- Motor can be started, stopped, and commanded to a target RPM.
- Closed-loop PID speed control demonstrated.
- Current, voltage, temperature, RPM, and fault status displayed locally.
- Faults detected and logged to flash.
- CAN commands can control the motor.
- Oscilloscope, logic analyzer, and JTAG evidence collected.
- Complete engineering documentation produced.
- Professional demo video recorded.

System Block Diagram



Selected Components

STM32G474: Primary MCU, RTOS, control algorithms

INA219: Current and voltage monitoring

MCP2515: SPI-based CAN controller

W25Q128: Event and fault logging

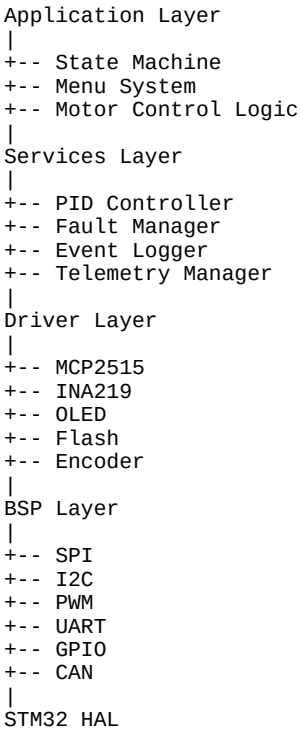
DRV8871: Motor driver

128x64 OLED: Local user interface

Rotary Encoder: User control

DC Gearmotor: Controlled physical plant

Firmware Architecture



RTOS Tasks

- Sensor Task (10ms)
- Control Task (10ms)
- Fault Task (20ms)
- Logger Task
- CAN Task
- Display Task
- CLI/Debug Task

2 Week Schedule

Day 1: Requirements, architecture diagrams, Git repository, folder structure, task planning.

Day 2: STM32CubeMX setup, FreeRTOS integration, UART debug console.

Day 3: Implement BSP layer: GPIO, SPI, I2C, UART, PWM.

Day 4: Implement INA219 driver. Validate current and voltage readings.

Day 5: Implement OLED driver and menu framework.

Day 6: Implement encoder support. Navigate menus and set RPM.

Day 7: Implement DRV8871 motor control. Open-loop PWM control.

Day 8: Add RPM measurement and encoder feedback. Verify measurements.

Day 9: Implement PID control loop. Tune for stable RPM regulation.

Day 10: Implement MCP2515 CAN driver and messaging protocol.

Day 11: Implement W25Q128 flash driver and event logging.

Day 12: Implement fault manager: overcurrent, sensor timeout, communication timeout.

Day 13: Perform debugging campaign. Capture oscilloscope, logic analyzer, and JTAG evidence.

Day 14: Record demo video. Complete validation report and documentation package.

Required Demonstration Sequence

1. Power-on self test.
2. Navigate OLED menus.
3. Set target RPM.
4. Start motor.
5. Show closed-loop RPM control.
6. Apply load by hand.
7. Demonstrate current limiting.
8. Trigger overcurrent fault.
9. Show fault LED indication.
10. Show fault log retrieval.
11. Disconnect sensor.
12. Demonstrate safe shutdown.
13. Send CAN command.
14. Show motor response.
15. Review captured debug evidence.

Required Debug Evidence

```
Oscilloscope:
- PWM duty cycle
- Motor startup current
- Current limiting event

Logic Analyzer:
- SPI transactions
- I2C transactions
- CAN packets

JTAG:
- Task inspection
- Queue inspection
- Breakpoint demonstration
```

Final Deliverables

1. Complete firmware repository
2. PCB schematic and layout
3. Requirements specification
4. Architecture document
5. Driver documentation
6. RTOS design document
7. CAN protocol specification
8. Validation report
9. Debug report
10. Demo video
11. Portfolio-ready screenshots and images